

Investigating Eigenvector Embeddings in PINNs

Alessandro Carraro (5523524), Javier Gallardo Saenz (6340474)

I. Introduction

Physics-informed neural networks (PINNs) have emerged as a powerful tool for solving partial differential equations (PDEs) by embedding the physical laws directly into the learning process [1]. Much research over the past year has highlighted several benefits of PINNs, including enhanced predictive accuracy with limited data, improved generalization, reduced dependence on large datasets, and a mesh-free formulation [2].

Despite these advantages, the common assertion that being mesh-free is an unqualified benefit requires further attention. Conventional PINNs typically rely on a Euclidean representation of the input space, where training is guided solely by minimizing the PDE residuals, without integrating explicit spatial domain information. This approach can lead to a misrepresentation of the true physical proximity or connectivity between points, particularly in bounded or irregular domains [3]. In fact, It has been widely reported that PINNs perform poorly on irregular domain [4], [5].

To address these challenges, Miao et al. [3] proposed augmenting the Euclidean input space with the Fiedler vector—the second smallest eigenvector of the graph Laplacian constructed from the computational mesh. This spectral embedding encodes intrinsic connectivity, capturing geodesic distances that Euclidean coordinates alone miss, especially in non-convex domains. For example, in a “C-shaped” domain, points close in Euclidean distance may be far apart along the boundary; the Fiedler vector helps the network distinguish such regions by providing a topological prior.

In this project, we investigate how incorporating eigenvectors as an additional input feature impacts PINN performance in solving the Poisson equation on complex 2D bounded domains. Specifically, does this spectral embedding improve solution accuracy and convergence compared to using Euclidean coordinates alone?

II. Methodology

A. Problem

We seek a neural network approximation $u_\theta : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}$ of the solution $u : \Omega \rightarrow \mathbb{R}$ to the elliptic boundary-value problem

$$-\Delta u(x) = f(x), \quad x \in \Omega, \quad u(x) = g(x), \quad x \in \Gamma_D, \quad \frac{\partial u}{\partial n}(x) = h(x), \quad x \in \Gamma_N, \quad (1)$$

where $\partial\Omega = \Gamma_D \cup \Gamma_N$ is a partition of the boundary into Dirichlet and Neumann portions with $\Gamma_D \neq \emptyset$, and $g : \Gamma_D \rightarrow \mathbb{R}$, $h : \Gamma_N \rightarrow \mathbb{R}$ are given. We make the assumption:

A1) Ω is a bounded piecewise smooth Lipschitz domain, $f \in L^2(\Omega)$, $l_{d-1}(\Gamma_D) > 0$

The weak formulation of the formulated Poisson problem reads as follows: "find $u \in x_0 + V$ such that

$$(\nabla u, \nabla v)_{L^2(\Omega)} = (f, v)_{L^2(\Omega)} \quad \forall v \in V \quad (2)$$

where $x_0 \in H^1(\Omega)$ satisfies the imposed boundary conditions and $V = \{v \in H^1(\Omega) \mid v|_{\Gamma_N} = 0 \text{ in the trace sense}\}$. Note that the Poisson equation on the given Ω trivially satisfies the conditions in [6, Theorem 4] and thus it has a unique solution for every $f \in L^2(\Omega)$.

B. Mode-wise learning

Given the Laplace–Beltrami (LB) operator $-\Delta$ on Ω with homogeneous boundary conditions (in our standard Euclidean case, the Laplace–Beltrami operator is just the Laplacian), its eigenvalue problem consists on finding a set of eigenfunctions $\{\phi_i\}_{i \in I}$ such that $-\Delta\phi_i = \lambda_i \phi_i \forall i \in I$. This yields the weak problem: "find $\{\phi_i\}_{i \in I}$ such that

$$(\nabla\phi_i, \nabla v)_{L^2} = \lambda_i(\phi_i, v)_{L^2} \quad \forall v \in V \quad (3)$$

Under the conditions of our problem, we can give stronger conditions on these Laplacian eigenfunctions. These are stated in the following Lemma:

Lemma II.1. *Under the assumption A1) and given homogeneous boundary conditions, the solution operator $A : L^2(\Omega) \rightarrow V \subset L^2(\Omega)$ defined as $Af = u$ (where u is the unique solution to 2 given f) has a set of eigenfunctions $\{\phi_i\}_{i=1}^\infty$ with positive eigenvalues $\{\mu_i\}_{i=1}^\infty$ such that $\mu_i \rightarrow 0$ as $i \rightarrow \infty$ which form an orthonormal basis of $L^2(\Omega)$. Moreover, it holds that $-\Delta\phi_i = \lambda_i\phi_i$ with $\lambda_i = \frac{1}{\mu_i} \forall i \in \mathbb{N}_{\geq 1}$*

It follows that any $u \in L^2(\Omega)$ can be expressed as an infinite series $u = \sum_{i=1}^\infty c_i \phi_i$ where $c_i = \langle u, \phi_i \rangle_{L^2(\Omega)}$. The proof of this lemma can be found in the Appendix. In this report, we work exclusively with homogeneous boundary conditions. However, when faced with a problem that has non-homogeneous boundary conditions, we can always find a function x_0 within our function space that satisfies the prescribed non-homogeneous conditions. We then subtract this x_0 from our unknown solution, defining a new unknown variable. This new variable will now satisfy homogeneous boundary conditions. This transformation effectively reduces the original problem to finding a solution in a function space where only homogeneous boundary conditions are present, precisely the space for which our eigenfunctions $\{\phi_i\}_{i=1}^\infty$ were derived.

In practice, we truncate to the first N modes and approximate $u \approx u_N = \sum_{i=1}^N c_i \phi_i$. Thus, each ϕ_i becomes a coordinate axis for frequency i , and the finite vector $(c_1, \dots, c_N)^\top$ encodes the projection of u onto the span of $\{\phi_i\}_{i=1}^N$.

C. PINNs

We employ a fully connected feedforward neural network $u_\theta : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}$, where in the base case the input consists of the spatial coordinates (x, y) and the network predicts the scalar output $u_\theta(x, y)$.

The embeddings are generated as follows: we construct a graph from the domain mesh generated by Netgen. Nodes correspond to the mesh vertices, and edges are formed between nodes that are directly connected in the mesh topology. Each edge is weighted using a Gaussian function of the Euclidean distance, $w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2}\right)$.

Since the neural network is evaluated at arbitrary points in the domain, we interpolate the eigenvector values across the mesh using piecewise linear interpolation over the mesh elements. This is not necessary if one uses LB eigenfunctions, though we observed that using the same interpolation strategy is more memory-efficient and does not have a negative impact on the results. In the augmented case, we enrich the network input by appending the first k nontrivial Laplacian eigenvectors (or the k first LB eigenfunctions) to the spatial coordinates. Each input point is mapped as:

$(x, y) \mapsto (x, y, \phi_2(x), \phi_3(x), \dots, \phi_{k+1}(x))^\top \in \mathbb{R}^{2+k}$ or $(x, y) \mapsto (x, y, \phi_1(x), \dots, \phi_k(x))^\top \in \mathbb{R}^{2+k}$ where $\phi_i(x)$ denotes the interpolated value of the i -th eigenvector or eigenfunction at location x , respectively.

Training All collocation points are drawn uniformly from the domain interior $\{x_j\}_{j=1}^{N_{\text{int}}} \subset \Omega$ and from the boundary $\{x_\ell\}_{\ell=1}^{N_{\text{bc}}} \subset \partial\Omega$. We minimize a weighted-mean-squared-error (MSE) objective:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} |\Delta u_\theta(x_j) - f(x_j)|^2,$$

$$\mathcal{L}_{\text{dir}}(\theta) = \frac{1}{N_{\text{bc}}^D} \sum_{\ell=1}^{N_{\text{bc}}^D} |u_\theta(x_\ell) - g(x_\ell)|^2, \quad \mathcal{L}_{\text{neu}}(\theta) = \frac{1}{N_{\text{bc}}^N} \sum_{m=1}^{N_{\text{bc}}^N} |\partial_n u_\theta(x_m) - h(x_m)|^2.$$

The combined loss is $\mathcal{L}(\theta) = w_{\text{pde}} \mathcal{L}_{\text{pde}}(\theta) + w_{\text{dir}} \mathcal{L}_{\text{dir}}(\theta) + w_{\text{neu}} \mathcal{L}_{\text{neu}}(\theta)$, with weights $w_{\text{pde}}, w_{\text{dir}}, w_{\text{neu}} > 0$. In each experiment, we sample both interior and boundary collocation points uniformly and train via standard gradient-based optimization to minimize $\mathcal{L}(\theta)$.

D. Finite Element Method ground truth

We use the Finite Element Method to compute an approximated solution of 2 -which we set as our ground-truth- and the LB eigenfunctions of 3. We use Netgen ([7]) to generate the meshes for our domains and NGSolve ([8]) to generate the results. Even though we do not consider non-homogeneous boundary conditions in this report, we present here the general approach to solving 2 with FEM.

The satisfaction of non-homogeneous Neumann conditions is ensured by including them in the RHS of the weak problem formulation, while the Dirichlet boundary conditions impose restrictions on our FE space. Thus, we solve the following FEM problem: "find $u_h \in \mathcal{F}_{\Gamma_D}(\mathbf{p}, \mathbf{k}; \tau_h)$ st

$$(\nabla u_h, \nabla v_h)_{L^2(\Omega)} = (f, v_h)_{L^2(\Omega)} - \int_{\Gamma_N} v_h g_N d(\Gamma_N) \quad \forall v \in \mathcal{F}_{\Gamma_D}(\mathbf{p}, \mathbf{k}; \tau_h) \quad "$$

where τ_h is a mesh of Ω , $\mathcal{F}(\mathbf{p}, \mathbf{k}; \tau_h)$ is our finite element space and $\mathcal{F}_{\Gamma_D}(\mathbf{p}, \mathbf{k}; \tau_h) \subset \mathcal{F}(\mathbf{p}, \mathbf{k}; \tau_h)$ denotes the FE space with the fixed degrees of freedom due to the Dirichlet boundary conditions.

On another hand, the LB eigenfunctions are always computed for homogeneous boundary conditions. Thus, we solve the the FE problem: "find $\phi_{i_h} \in \mathcal{F}_{\Gamma_{D_h}}(\mathbf{p}, \mathbf{k}; \tau_h)$ st

$$(\nabla \phi_{i_h}, \nabla v_h)_{L^2(\Omega)} = \lambda_i (\phi_{i_h}, v_h)_{L^2(\Omega)} \quad \forall v \in \mathcal{F}_{\Gamma_{D_h}}(\mathbf{p}, \mathbf{k}; \tau_h) \quad "$$

where $\mathcal{F}_{\Gamma_{D_h}}(\mathbf{p}, \mathbf{k}; \tau_h) \subset \mathcal{F}(\mathbf{p}, \mathbf{k}; \tau_h)$ denotes the FE space with the fixed degrees of freedom due to the homogeneous Dirichlet boundary conditions. The bilinear forms on both sides of the equation give rise to two FE matrices, let us name them B for the LHS and M for the RHS. Thus, after assembly, the problem reduces to the eigenvalue problem $Bx = \lambda Mx$. This is then solved by means of the PINVIT algorithm ([9]).

E. Mode-Wise Coefficient Comparison

After training the PINN to produce the approximation $u_\theta(x) = \mathcal{N}_\theta(x)$, we predict u_θ over a uniform grid of interior points $\{x_j\}_{j=1}^n$ and assemble the vector $u_\theta = (u_\theta(x_1), \dots, u_\theta(x_n))^T$. To compare the learned and true solutions, we project both onto a set of eigenfunctions $\{\phi_i\}_{i=1}^N$ to obtain their respective spectral coefficients. Specifically, the i -th spectral coefficient of a function $u \in L^2(\Omega)$ is computed as $c_i = (u, \phi_i)_{L^2(\Omega)}$. When u is an NGSolve function we compute this inner product in NGSolve for increased efficiency. However, due to incompatibility issues between NGSolve and numpy/jax, the inner products for the outputs of the Neural

Network need to be computed as:

$$(u_\theta, \phi_i)_{L^2(\Omega)} \approx \frac{\ell(\Omega)}{n} \sum_{j=1}^n \phi_i(x_j) u_\theta(x_j), \quad i = 1, \dots, N, \quad (4)$$

where $\ell(\Omega)$ is the area of the domain Ω . Applying this procedure to both the PINN solution u_θ and the FEM solution $u^\star$ yields the learned coefficients $\{\hat{c}_i\}$ and the true coefficients $\{c_i\}$, respectively. This enables a direct mode-wise comparison between the learned and true solutions, regardless of the specific source function.

Manufactured Eigenbasis Solution To analyse the learning dynamics under specified spectral energy distribution we can construct a manufactured solution of the form $u^\star(x) = \sum_{i=1}^N c_i \phi_i(x)$ with prescribed $\{c_i\}$. In that case the generated source term is $f(x) = -\Delta u^\star(x) = \sum_{i=1}^N \lambda_i c_i \phi_i(x)$. Training the PINN on this manufactured f and then projecting u_θ onto $\{\phi_i\}$ allows direct comparison of each $\hat{c}_i$ against the known c_i , thereby quantifying mode-wise learning behavior under controlled spectral energy distributions.

F. Experiments

We compare PINNs using eigenvector embeddings with $k = 0, 1, 2, 3, 10$ eigenvectors on the following geometries, each chosen for its particular challenge:

- **C-section:** A standard non-convex domain used in prior work by Miao [3].
- **Dumbbell:** Two large lobes connected by a narrow neck, testing the PINN’s ability to propagate information through bottlenecks.
- **Polygonal Spiral:** Requires the PINN to follow intrinsic geodesic distances along the spiral.

We evaluated several source functions, primarily the spectral expansion from man sol and the sinusoidal function $f(x, y) = \sin(\pi x) \sin(\pi y)$ supported on a 1×1 block and zero elsewhere. This support was chosen to promote diffusion aligned with the domain’s geodesic structure. Boundary conditions were zero Neumann everywhere except on the southern edge, where zero Dirichlet conditions induced a gradient: $\frac{\partial u}{\partial n} = 0$ on $\partial\Gamma \setminus \Gamma_{\text{south}}$, $u = 0$ on Γ_{south} .

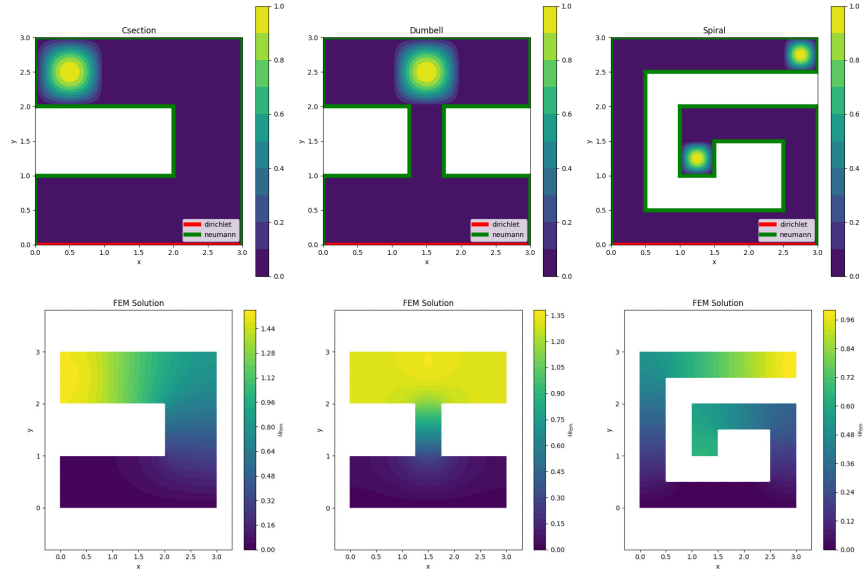


Figure 1. Sinusoidal source test function and Ground truth FEM solutions

III. Results

An initial grid search was conducted to identify a suitable network architecture, activation function, and training regime. The optimal configuration was found to use a $\tanh$ activation function, a learning rate of 1×10^{-3} , 10,000 training epochs, and a network architecture with layers $[2 + k, 64, 64, 64, 1]$ with biases. As noted by Krishnapriyan et al. [10], PINNs are particularly sensitive to the relative weighting of loss terms; improper weight selection can lead to training instability and suboptimal solutions. We observed similar sensitivities in our experiments and therefore performed a second grid search to tune the loss weights w_{pde} , w_{neu} , and w_{dir} for our specific problem.

A. Sine Source Function

The network performance is evaluated using two metrics: the relative L^2 error and the relative L^∞ error. The relative L^2 error is defined as

$$e_{L^2} = \frac{\|u_\theta - u^\star\|_{L^2}}{\|u^\star\|_{L^2}},$$

where u_θ is the predicted solution and $u^\star$ is the exact solution. The error is computed over a dense 300×300 evaluation grid, considering internal points only.

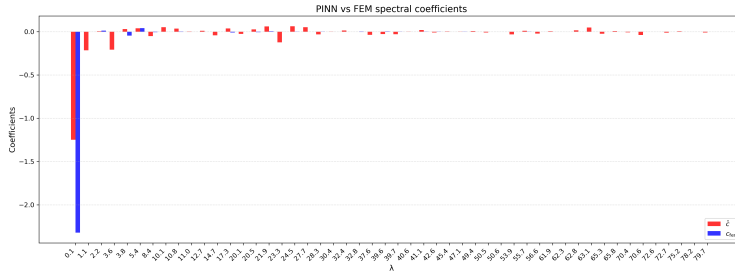
Table 1. Best results per k for CSECTION, DUMBELL, and SPIRAL. Grid search performed over $w_{\text{pde}} \in \{1, 5, 10, 50, 100\}$, $w_{\text{neu}} \in \{1, 5, 10, 50\}$ and fixed $w_{\text{dir}} = 0.3$

k	CSECTION				DUMBELL				SPIRAL			
	Eigenvectors		Eigenfunctions		Eigenvectors		Eigenfunctions		Eigenvectors		Eigenfunctions	
	L2 Error	L^∞ Error	L2 Error	L^∞ Error	L2 Error	L^∞ Error	L2 Error	L^∞ Error	L2 Error	L^∞ Error	L2 Error	L^∞ Error
0	0.8637	1.2252	0.8637	1.2252	0.9740	1.3399	0.9740	1.3399	0.9740	1.3399	0.9740	1.3399
1	1.1192	1.9637	0.7157	1.1449	0.9814	1.3542	0.9835	1.6029	0.9814	1.3542	0.7153	0.71822
2	1.1478	1.8051	1.2289	2.5105	1.0064	1.3951	0.5731	0.9077	1.0064	1.3951	1.4644	1.4480
3	1.0756	1.7488	0.8646	1.3038	1.0772	1.8346	0.4921	0.9061	1.0772	1.8346	1.6210	1.7970
10	0.9874	1.5551	1.1839	2.2344	0.9930	1.4445	0.9251	2.1807	0.9930	1.4445	1.3815	1.2560

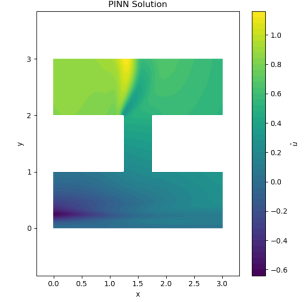
From Table 1, we observe that the L^2 and L^∞ errors never fall below those of the baseline Euclidean model when using the discrete Laplacian eigenvectors; in fact, they often worsen. While the use of LB eigenfunctions does seem to improve the results, it does not manage to produce meaningful predictions of the ground truth, and the addition of too many eigenfunctions once again worsens the model’s performance. This suggests a more fundamental limitation in the network’s learning dynamics, rather than a deficiency in encoding geodesic distances. The intrinsic geometry is already enforced through uniform interior and boundary sampling, combined with appropriately chosen PDE and Neumann weights, as illustrated in Figure 3.

The spectral analysis further supports this conclusion. By projecting both the FEM solution and the PINN output onto the Laplace–Beltrami basis, we consistently find that the coefficient corresponding to the first eigenmode is severely under-recovered. This issue persists across all experiments and appears to be a key driver of the network’s poor performance.

Throughout training, the loss decreases very slowly, and the first eigenmode coefficient is never effectively learned. Even after 10,000 epochs, the network primarily captures only the smaller, higher-frequency coefficients.

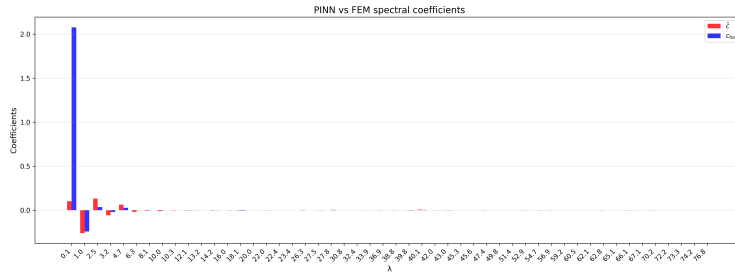


(a) Spectral coefficients comparison.

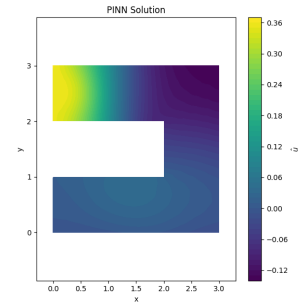


(b) PINN prediction.

Figure 2. Spectral coefficients and network prediction on DUMBELL for the best-performing model ($k = 3$ eigenfunctions). The blue bars correspond to the spectral coefficients of the FEM solution, while the red bars show the spectral coefficients of the PINN prediction. The right panel shows the PINN solution.



(a) Spectral coefficients comparison.



(b) PINN prediction.

Figure 3. Spectral coefficients and network prediction on CSECTION for the PINN with $k = 0$. The blue bars correspond to the spectral coefficients of the FEM solution, while the red bars show the spectral coefficients of the PINN prediction. The right panel shows the PINN solution.

B. Spectral Source Function

We then modified the problem to use a purely spectral source function, where the spectral coefficients were explicitly prescribed by the sequence: $c_i = \frac{i}{\lambda_{\max}}$, $i = 1, \dots, 50$, such that higher-frequency modes were assigned more energy. This setup intentionally emphasizes the high-frequency components of the solution, in contrast to the original case where the solution energy was primarily concentrated in the low-frequency modes. To complement this, we also considered the converse scenario, where the low-frequency modes were dominant and the spectral coefficients decreased with increasing mode number, thus prioritizing low-frequency energy.

Due to time constraints and based on the previous results, we restricted our analysis to the cases $k = 0$ and $k = 1$ on the CSECTION, focusing on whether the previously observed mismatch in the low-frequency coefficients would persist under this new regime. The visual results can be found in Figure 7

C. Regularization

To further investigate the persistent under-recovery of low-frequency spectral components, we performed an additional experiment by introducing a targeted regularization on the first spectral coefficient. We did this using two different methods: a weak regularization and a strong regularization.

Weak-regularization - we compute the first LB coefficient of the solution, $c_1^\star$, by projecting the source function f onto the first Laplace–Beltrami eigenfunction ϕ_1 and dividing by λ_1 . We then augmented the physics-informed loss with a regularization term that penalizes the deviation of the network’s estimated first coefficient from this true value. The modified loss function is written as:

$$\mathcal{L}_{\text{PINN}} = w_{\text{pde}} \mathcal{L}_{\text{pde}} + w_{\text{neu}} \mathcal{L}_{\text{n}} + w_{\text{dir}} \mathcal{L}_{\text{dir}} + w_{\text{reg}} \left\| (\phi_1, u_\theta)_{L^2(\Omega)} - c_1^\star \right\|^2$$

k-term Strong-regularization - we compute the k-first LB coefficients of the solution, $\{c_1^\star, \dots, c_k^\star\}$, as $c_i^\star = \frac{(\phi_i, f)_{L^2(\Omega)}}{\lambda_i}$. Then, we learn the residual of the spectral expansion $u_k = \sum_{i=1}^n c_i^\star \phi_i$, so the PDE loss becomes

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} \left| \sum_{i=1}^n \lambda_i c_i^\star \phi_i - \Delta r_\theta(x_j) - f(x_j) \right|^2$$

and the regularization loss imposes that the residual is orthogonal to the span of the first k LB functions, $w_{\text{reg}} \mathcal{L}_{\text{reg}} = w_{\text{reg}} \left\| \sum_{i=1}^k (\phi_i, r_\theta)_{L^2(\Omega)} \right\|^2$.

The eigenfunctions ϕ_i are precomputed and treated as constants with respect to the network parameters. This ensures that the inner product $(u_\theta, \phi_i)_{L^2(\Omega)}$ is differentiable, as the gradients only need to flow through u_θ (same applies for r_θ).

By introducing this spectral regularization, we directly force the network to capture the correct energy in the most significant low-frequency mode. Nevertheless, the learning dynamics induced by each of the two proposed regularization methods are intrinsically different, as the weak regularization evaluates the capacity of the Neural Network of learning the solution when it is aided by the knowledge of the first coefficient; while the strong regularization evaluates the capacity of the Neural Network to learn the residual of a spectral expansion.

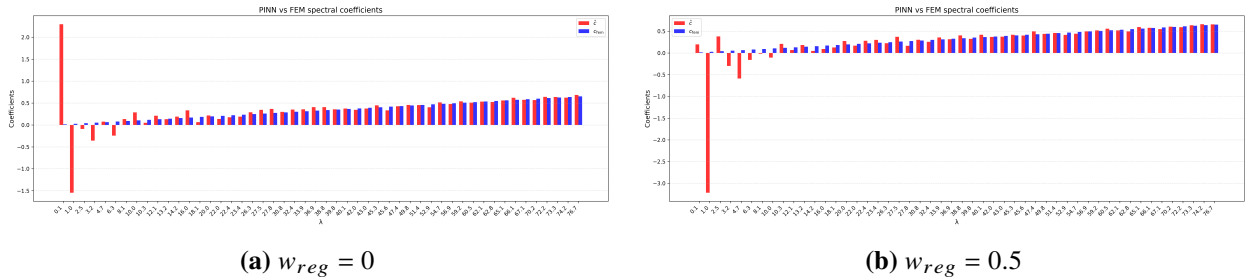


Figure 4. Spectral coefficients for (a) $w_{\text{reg}} = 0$ and (b) $w_{\text{reg}} = 0.5$ on the CSECTION with $k = 0$. When regularization is applied, the first coefficient learns the correct magnitude, but the error increases in the second coefficient.

Unsurprisingly, the baseline 1-term spectral expansion outperformed all proposed 1-term regularization methods in the manufactured spectral problem with ascending coefficients. This is because regularization provides little information to the Neural Network in such cases. However, both regularization methods significantly outperformed the 1-term spectral expansion in the descending problem, where knowledge of the first coefficient acts as a strong prior. We found that embedding with eigenvectors or eigenfunctions, when regularization was used, generally worsened performance, offering no conclusive results. Interestingly, the best result for the descending sequence still incorporated one embedding.

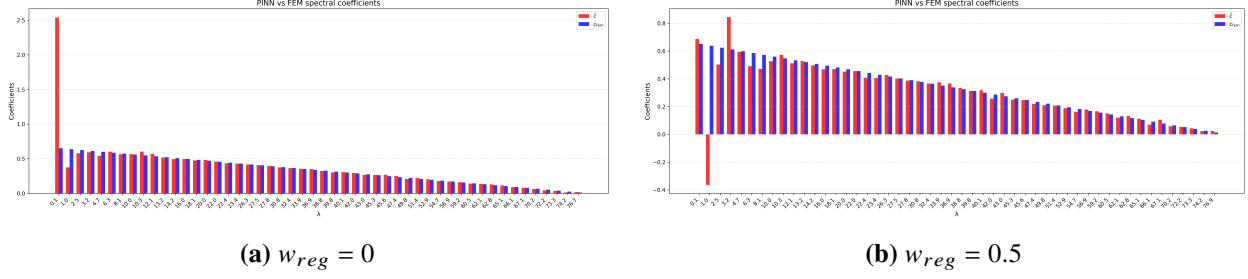


Figure 5. Spectral coefficients for (a) $w_{reg} = 0$ and (b) $w_{reg} = 0.5$ on the CSECTION with $k = 0$. When regularization is applied, the first coefficient learns the correct magnitude, but the error increases in the second coefficient.

Table 2. Comparison of L^2 and L^∞ errors for ascending and descending spectral ordering. Best performances per ordering are highlighted in bold. All runs use $w_{pde} = 1.5$, $w_{dir} = 0.3$, $w_{neu} = 1.0$ and $w_{reg} = 0.5$ (when regularization is applied), trained over 10 000 epochs

Regularization	k -vectors	Ascending		Descending	
		L^2 Error	L^∞ Error	L^2 Error	L^∞ Error
No Regularization	0	1.0542	2.7925	0.7064	1.1426
Weak Regularization	0	1.2511	4.8841	0.3930	0.7917
Strong Regularization (1-term)	0	1.0666	2.8672	0.74546	3.6010
No Regularization	1	2.5512	4.5366	0.5929	1.3638
Weak Regularization	1	1.0365	5.0022	0.8061	2.3116
Strong Regularization (1 term)	1	1.5975	3.4503	0.2903	1.0890
1-term Spectral Expansion	-	1.0000	1.0000	0.9999	0.9999

What is perhaps more shocking is that the strong regularization outperforms the 2-term spectral expansion baseline when given c_2 as a prior as well as c_1 for the ascending problem, even though c_2 still carries remarkably little information about the problem. The 2-term strong regularization achieved an L^2 relative error of **0.3226** versus the 0.9999 L^2 relative error of the baseline. Additionally, we observed that biasing the model with the first two spectral coefficients caused the error in the third coefficient to surge in comparison with the observed error on the third coefficient under a 1-term regularization - perpetuating the effect observed in Figure 4.

Adding regularization to the Neural Network is, arguably, a form of "cheating," but it is nonetheless relevant to observe the dynamics of the model once the first coefficient is provided, given that this first coefficient represents the primary error mode. However, once one has made the effort to pre-compute some LB functions for a given domain and boundary conditions -and given that the spectral coefficients of the solution are easy to compute without knowledge of the solution, as is the case here- it is interesting to see whether the Neural Network can act as "error-smoother" by significantly reducing the error made by the computed spectral expansion in a short number of epochs.

Indeed, the results were positive: the strong regularization method significantly smoothed the error in only 500 epochs across all domains and source functions with a significant weight in the first spectral components. Due to space constraints we do not present the full results of our experiments here, but -taking the CSECTION with the default sinusoidal bump source as an example- the 1-term strong regularization achieved an L^2

relative error of **0.0365** after only 500 epochs, which is a great improvement over the $0.1178 L^2$ relative error of the 1-term spectral expansion on the same problem, and reasonably close to the $0.0245 L^2$ relative error of the 2-term spectral expansion. The weak regularization problem, however, was not able to outperform the 1-term spectral expansion after only 500 epochs, and its L^2 relative error after 10000 epochs was still 0.0776.

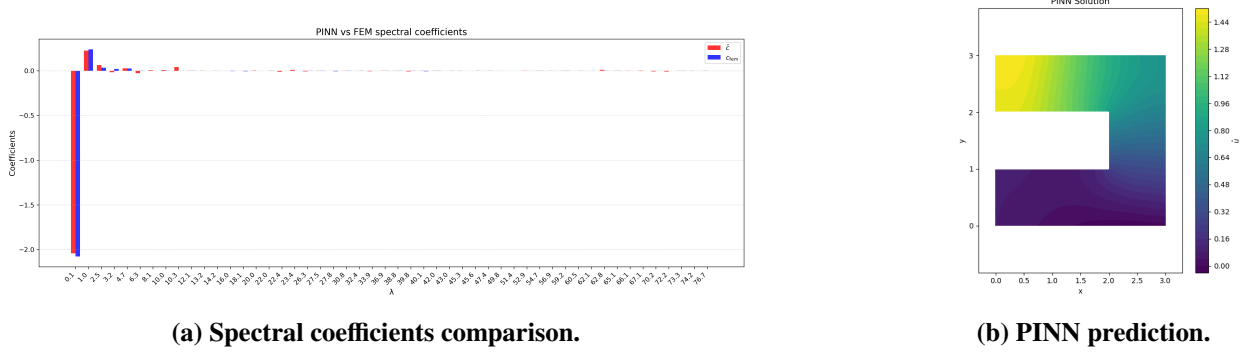


Figure 6. Spectral coefficients and network prediction on CSECTION with sinusoidal bump source for the 1-term strong regularization. The blue bars correspond to the spectral coefficients of the FEM solution, while the red bars show the spectral coefficients of the PINN prediction. The right panel shows the PINN solution.

IV. Discussion

A. The role of the higher order vectors

It is well known that the Fiedler vector encodes the mesh’s intrinsic connectivity so explaining the inconsistent performance can be complicated and would require further attention. On the other hand, for higher eigenvectors we can make some hypothesis. First notice their correspondence with the true Laplace–Beltrami modes weakens as the eigenvalue index increases. Furthermore, mesh refinement may not alleviate this issue. As shown by Belkin et al. [11], for manifolds with boundaries, increasing mesh refinement causes discrete graph eigenvectors to converge to eigenfunctions that satisfy zero Neumann boundary conditions, regardless of the actual PDE boundary conditions being Dirichlet or mixed. This phenomenon could explain why higher-order eigenvectors might degrade performance by introducing inaccurate or misleading information.

It is also worth noting that performing PCA on the feature matrix $X = [x, y, \phi_2, \dots, \phi_{k+1}] \in \mathbb{R}^{N \times (2+k)}$, we observe that most of its singular values are effectively zero, with only the first two being significant. This indicates extreme collinearity among the features. While this is a well-known issue in OLS regression, its impact on neural networks is less clear. Some authors [12] argue that, because neural networks are typically highly overparameterized, the additional weights that arise from collinear features are unlikely to introduce performance degradation. In other words, the network’s inherent redundancy means that the influence of these collinear feature should be minimal. Nevertheless, there remains no consensus on the precise effect of multicollinearity in neural network training.

B. On the challenge of learning the 1st Laplace-Beltrami eigenmode

To keep notation light and ensure direct comparability with our experiments, suppose that we are working with homogeneous boundary conditions (the results presented here, however, generalize to non-homogeneous bcs). Assume that, after training, we get $-\Delta u_\theta = f + \epsilon$ (with $\epsilon \in L^2(\Omega)$), so that $\mathcal{L}_{PDE} \approx \frac{1}{l(\Omega)} \|\epsilon\|_{L^2}$. This means

that $(\nabla u_\theta, \nabla v)_{L^2} = (f + \epsilon, v)_{L^2} = (\nabla A f + \nabla A \epsilon, \nabla v)_{L^2} = (\nabla(u + A \epsilon), \nabla v)_{L^2}$. Thus, $(\nabla(u_\theta - u - A \epsilon), \nabla v)_{L^2} = 0$ $\forall v \in V$ which (as long as $u_\theta \in V$, which is a reasonable assumption because the Dirichlet loss is extremely low in all of our experiments) gives us $\nabla(u_\theta - u) = \nabla A \epsilon$.

Let $e = u_\theta - u$. Clearly $e \in L^2(\Omega)$ and thus we can take its expansion $e = \sum_{i=1}^{\infty} e_i \phi_i$ where $e_i = (e, \phi_i)_{L^2}$. Note that we can also take the expansion $\epsilon = \sum_{i=1}^{\infty} \epsilon_i \phi_i$ where again $\epsilon_i = (\epsilon, \phi_i)_{L^2}$. Thus, we have that $A \epsilon = \sum_{i=1}^{\infty} \mu_i \epsilon_i \phi_i$ and therefore $\nabla A \epsilon = \sum_{i=1}^{\infty} \lambda_i \mu_i \epsilon_i \phi_i = \sum_{i=1}^{\infty} \epsilon_i \phi_i$. Furthermore, observe that $\nabla e = \sum_{i=1}^{\infty} \lambda_i e_i \phi_i$. Putting all this together:

$$\nabla(u_\theta - u) = \nabla A \epsilon \Rightarrow \sum_{i=1}^{\infty} \lambda_i e_i \phi_i = \sum_{i=1}^{\infty} \epsilon_i \phi_i \Rightarrow e_i = \frac{1}{\lambda_i} \epsilon_i = \mu_i \epsilon_i \quad \forall i \in \mathbb{N}_{\geq 1} \quad (5)$$

This means that the projection of the true error $u_\theta - u$ to the Laplace-Beltrami basis functions is given by the projection of the PDE loss error to the Laplace-Beltrami basis **amplified** by the inverse of the first Laplace-Beltrami eigenvalue.

This means that if we have a small spectral gap (i.e. small λ_1 , which is exactly what we have observed) and a slowly varying PDE loss error (which implies a high e_1), the true error in the first eigenmode of the Laplace Beltrami ONB is highly amplified. For reference, the first Laplace-Beltrami eigenvalue is 0.1224 in the CSECTION, 0.0941 in the DUMBELL and 0.0898 in the SPIRAL.

What is more, the PDE loss error in higher frequencies gets damped because $\lambda_i \rightarrow \infty$ as $i \rightarrow 0$. This observation perfectly matches the results of our experiments and may help explain the remarkable performance of the 1-term strong regularization method. Regrettably, due to time constraints, we have not been able to obtain conclusive results about the use of embeddings in the strong regularization

V. Conclusion

Contrary to the findings in [3] and [13], our PINN's performance did not significantly improve when embedded with Laplacian eigenvectors or Laplace-Beltrami eigenfunctions in the context of our complex geometries. This suggests that the benefits of these embeddings may be context-dependent and not universally applicable to all PDE problems in complicated geometries.

Even without directly implementing the architecture described in [5], the observed effectiveness of a simple spectral expansion begs caution. Our results suggest that if pre-computing multiple Laplace-Beltrami eigenfunctions is feasible, learning the residual of their spectral expansion could be enough to achieve high performance. Given the strength of Laplace-Beltrami eigenfunctions, exploring suitable orthonormal bases that complement the PINN model could be a promising avenue.

Our observations regarding frequency learning within our PINN diverge from the typical scenario in PINNs and broader machine learning contexts, where higher-order frequencies are generally more challenging to learn, and lower-order frequencies are effectively recovered ([14], [15], [16]). As elaborated in Equation 5, these differences do not stem from a model deficiency but rather from an intrinsic relationship between the PDE Loss and the eigenmodes of the Laplace-Beltrami eigenfunctions. This intriguing result appears to be potentially generalizable to any PDE governed by an operator satisfying the conditions in Lemma II.1. This interesting finding warrants further investigation.

References

- [1] Raissi, M., Perdikaris, P., and Karniadakis, G., “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, Vol. 378, 2019, pp. 686–707. doi: <https://doi.org/10.1016/j.jcp.2018.10.045>, URL <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [2] Cuomo, S., Cola, V. S. D., Giampaolo, F., Rozza, G., Raissi, M., and Piccialli, F., “Scientific Machine Learning through Physics-Informed Neural Networks: Where we are and What’s next,” *CoRR*, Vol. abs/2201.05624, 2022. URL <https://arxiv.org/abs/2201.05624>.
- [3] Miao, Y., Li, H., and Mandic, D., “GPINN: Physics-informed Neural Network with Graph Embedding,” 2023. doi: 10.48550/arXiv.2306.09792.
- [4] Goswami, S., Anitescu, C., Chakraborty, S., and Rabczuk, T., “Transfer learning enhanced physics informed neural network for phase-field modeling of fracture,” *Theoretical and Applied Fracture Mechanics*, Vol. 106, 2020, p. 102447. doi: <https://doi.org/10.1016/j.tafmec.2019.102447>, URL <https://www.sciencedirect.com/science/article/pii/S016784421930357X>.
- [5] Sahli Costabal, F., Pezzuto, S., and Perdikaris, P., “-PINNs: Physics-informed neural networks on complex geometries,” *Engineering Applications of Artificial Intelligence*, Vol. 127, 2024, p. 107324. doi: <https://doi.org/10.1016/j.engappai.2023.107324>, URL <https://www.sciencedirect.com/science/article/pii/S0952197623015087>.
- [6] Karátson, J., and Korotov, S., “Discrete maximum principles for finite element solutions of nonlinear elliptic problems with mixed boundary conditions,” *Numerische Mathematik*, Vol. 99, No. 4, 2005, pp. 669–698. doi: 10.1007/s00211-004-0559-0.
- [7] Schöberl, J., “NETGEN An advancing front 2D/3D-mesh generator based on abstract rules,” *Computing and Visualization in Science*, Vol. 1, No. 1, 1997, pp. 41–52.
- [8] Schöberl, J., “C++11 Implementation of Finite Elements in NGSolve,” *ASC Report*, Vol. 30, 2014, pp. 1–23. URL <http://hdl.handle.net/20.500.12708/28346>.
- [9] Neymeyr, K., “A geometric theory for preconditioned inverse iteration II: Convergence estimates,” *Linear Algebra and its Applications*, Vol. 322, No. 1, 2001, pp. 87–104. doi: [https://doi.org/10.1016/S0024-3795\(00\)00236-6](https://doi.org/10.1016/S0024-3795(00)00236-6), URL <https://www.sciencedirect.com/science/article/pii/S0024379500002366>.
- [10] Krishnapriyan, A. S., Gholami, A., Zhe, S., Kirby, R. M., and Mahoney, M. W., “Characterizing possible failure modes in physics-informed neural networks,” , 2021. URL <https://arxiv.org/abs/2109.01050>.
- [11] Belkin, M., Que, Q., Wang, Y., and Zhou, X., “Graph Laplacians on Singular Manifolds: Toward understanding complex spaces: graph Laplacians on manifolds with singularities and boundaries,” *CoRR*, Vol. abs/1211.6727, 2012. URL <http://arxiv.org/abs/1211.6727>.
- [12] De, R., Lyle, V., and Ungar, H., “Multicollinearity: A tale of two nonparametric regressions,” 2000. doi: 10.1007/978-1-4612-2660-4_0.
- [13] Li, H., Miao, Y., Khodaei, Z. S., and Aliabadi, M. H., “FINITE-PINN: A Physics-Informed Neural Network with Finite Geometric Encoding for Solid Mechanics,” *arXiv preprint arXiv:2412.09453*, 2024. URL <https://arxiv.org/abs/2412.09453>.
- [14] Jacot, A., Gabriel, F., and Hongler, C., “Neural Tangent Kernel: Convergence and Generalization in Neural Networks,” *Advances in Neural Information Processing Systems*, Vol. 31, 2018.
- [15] Wang, S., Teng, A., and Perdikaris, P., “Understanding and Mitigating Spectral Bias in Physics-Informed Neural Networks,” *Journal of Computational Physics*, Vol. 435, 2021, p. 110292.
- [16] Rahaman, N., Baratin, A., Valiant, D., Goyal, A., Khan, H., Berthet, Q., Kasai, M., Larochelle, H., Manzagol, D., Bengio, Y., et al., “Spectral bias in deep neural networks,” *arXiv preprint arXiv:1901.06082*, 2019.
- [17] Ziemer, W. P., *Weakly Differentiable Functions*, Graduate Texts in Mathematics, Vol. 120, Springer-Verlag, New York, 1989.

Appendix

Lemma V.1. *Under the assumption A1) and given homogeneous boundary conditions, the solution operator $A : L^2(\Omega) \rightarrow V \subset L^2(\Omega)$ defined as $Af = u$ (where u is the unique solution to 2 given f) has a set of eigenfunctions $\{\phi_i\}_{i=1}^\infty$ with positive eigenvalues $\{\mu_i\}_{i=1}^\infty$ such that $\mu_i \rightarrow 0$ as $i \rightarrow \infty$ which form an orthonormal basis of $L^2(\Omega)$. Moreover, it holds that $-\Delta\phi_i = \lambda_i\phi_i$ with $\lambda_i = \frac{1}{\mu_i} \forall i \in \mathbb{N}_{\geq 1}$*

Proof. The Poincare inequality can be generalized to our case with mixed boundary conditions as long as $l_{d-1}(\Gamma_N) > 0$ (see [17, Section 4.5]). For any $f, g \in L^2(\Omega)$, we have that $(f, Ag)_{L^2} = (\nabla Af, \nabla Ag)_{L^2} = (Af, g)_{L^2}$ so the operator is symmetric. Note also that $(f, Af)_{L^2} = (\nabla Af, \nabla Af)_{L^2}$ so together with the generalized Poincare inequality we get coercivity and boundedness of A . A is injective due to uniqueness. Finally, note that V is dense in $L^2(\Omega)$ and thus boundedness and the Rellich-Kondrachov theorem gives us compactness of A . Reasoning with the extended operator $\tilde{A} : V^* \rightarrow V$ and using density we get that $R(A)$ is dense in $L^2(\Omega)$.

Therefore, by the Spectral Theorem for compact and symmetric operators there exists an ONB $\{\phi_i\}_{i=1}^\infty$ of $L^2(\Omega)$ st $A\phi_i = \mu_i\phi_i$ with positive (due to coercivity) $\mu_i \rightarrow 0$ as $i \rightarrow \infty$.

Finally, note that for any $v \in V$, $i \in \mathbb{N}_{\geq 1}$ we have: $(\phi_i, v)_{L^2} = (\nabla A\phi_i, \nabla v)_{L^2} = \mu_i(\nabla\phi_i, \nabla v)_{L^2} \Rightarrow (\nabla\phi_i, \nabla v)_{L^2} = \frac{1}{\mu_i}(\phi_i, v)_{L^2}$. This is exactly 3 and thus $\{\phi_i\}_{i=1}^\infty$ are also eigenfunctions of $-\nabla$ with eigenvalues $\lambda_i = \frac{1}{\mu_i} \rightarrow \infty$ as $i \rightarrow \infty$

□

□

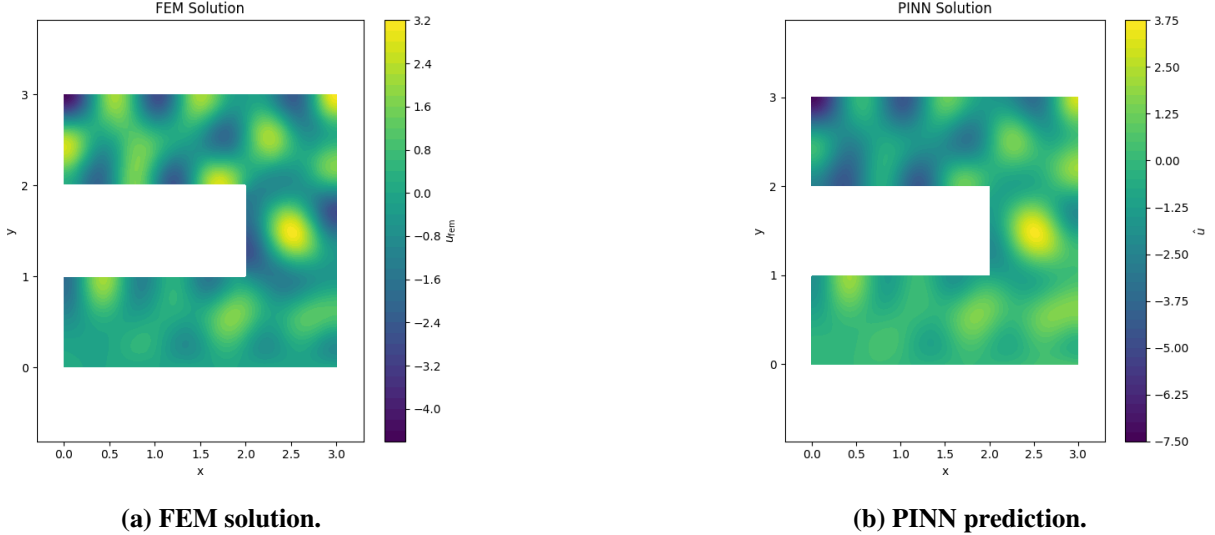


Figure 7. Comparison of the FEM solution and the PINN prediction on the manufactured spectral source function. The L^2 error 1.0365